

EE2703: Assignment_3

Vikas Raj (EE19B108)

1 Abstract

In this assignment we aim to :

- Observe the error in fitting the *Least Error Fit* function to a given set of data.
- Find the relation between the error observed and the noise in the data.

2 Introduction

From linear algebra, we can condense any parameter estimation problem to a simple matrix equation of the form:

$$\begin{pmatrix} F_1(t_1) & F_2(t_1) & \dots & F_n(t_1) \\ F_1(t_2) & F_2(t_2) & \dots & F_n(t_2) \\ \dots & \dots & \dots & \dots \\ F_1(t_m) & F_2(t_m) & \dots & F_n(t_m) \end{pmatrix} \begin{pmatrix} p_1 \\ p_2 \\ \dots \\ p_n \end{pmatrix} = \begin{pmatrix} a_1 \\ a_2 \\ \dots \\ a_m \end{pmatrix} \quad (1)$$

where,

$$f(t; p_1, p_2, \dots, p_n) = \sum_{i=1}^n p_i F_i(t) \quad (2)$$

is the function to be estimated, $F_i(t)$ are arbitrary functions of the variable t and p_i are constant parameters.

Since we only have to “fit” the real-time data we have to a function, $F_i(t)$ are functions of our choice. Usually we make a few educated guesses for these functions, looking at the plots of these real-time data.

Equation (1) can be written as:

$$F \cdot \vec{p} = \vec{a}_0 \quad (3)$$

But, in any real world situation, there will always be noise associated with the data. To account for that, we have to slightly modify (3) to:

$$F \cdot \vec{p} = \vec{a}_0 + \vec{n} = \vec{a} \quad (4)$$

where $\vec{n}$ accounts for the noise in the data.

However, the above equation cannot be satisfied exactly always (as number of equations is $\mathbf{N}$, but the number of measurements is $\mathbf{M}$).

So, we make a few assumptions about the noise in the data and then try to “best guess” the solution, i.e., the error between the ideal solution, in the absence of noise and the one obtained has to be as minimum as possible.

The assumptions we make about the noise are that it has zero mean and a standard deviation of σ .

The error function ϵ is then given by:

$$\epsilon = F \cdot \vec{p} - \vec{a} \quad (5)$$

The norm of the error is:

$$\|\epsilon\|^2 = \sum_i \epsilon_i^2 \quad (6)$$

or in other words,

$$\|\epsilon\|^2 = \epsilon^T \epsilon = ((F \cdot \vec{p} - \vec{a})^T (F \cdot \vec{p} - \vec{a})) = \sum_i \epsilon_i^2 \quad (7)$$

On expanding the above equation and using the condition that the gradient at the minima is 0, we get:

$$\begin{aligned} 2(F^T F) \vec{p}_0 - 2F^T \vec{a} &= 0 \\ \implies \vec{p}_0 &= (F^T F)^{-1} F^T \vec{a} \end{aligned} \quad (8)$$

The above result is called the ***Least Squares Estimate of $\vec{p}_0$*** .

However, the above result holds true only if the initial functions $F_i(t)$ are independent and that the noise is same for different measurements. This is because, we have to *give lesser importance (**weight**) to those values which have more noise*, as they are more unreliable.

3 Procedure

The function to be fitted is:

$$f(t) = 1.05J_2(t) - 0.105t \quad (9)$$

where $J_2(t)$ is the *Bessel Function of the first kind of Order 2*. The true data used for fitting is obtained using this equation.

3.1 Creating noisy data

To create the noisy data, we add random noise to $f(t)$. This random noise, denoted by $n(t)$, is given by the standard normal probability distribution:

$$P(n(t)|\sigma) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{n(t)^2}{2\sigma^2}} \quad (10)$$

The resulting noisy data will be of the form:

$$f(t) = 1.05J_2(t) - 0.105t + n_{\sigma_i}(t) \quad (11)$$

where, $n_{\sigma_i}(t)$ is the noisy data function with $\sigma = \sigma_i$ in (10). Thus for 9 different values of sigma (in a log scale from 0.001 to 0.1), the noisy data is created and stored in the `fitting.dat` file.

3.2 Analyzing the noisy data

The data is read and plotted using PyPlot. The output result looks as follows:

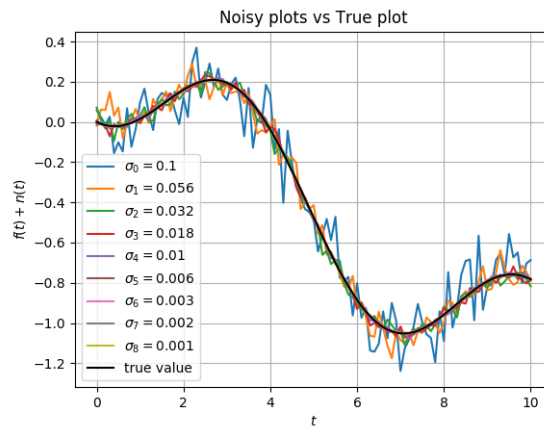


Figure 1: Noisy Data with True Data

As we can see, the “noisiness” of the data increases with increasing value of σ . Another view of how the noise affects the data can be seen below:

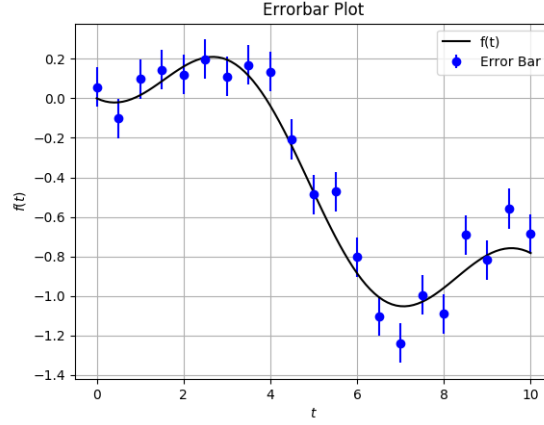


Figure 2: Noisy Data with Errorbar

The blue lines (*error bar*) indicate the standard deviation of the noisy data from the original data, at that value of t . It is plotted at every 5th point to make the plot readable.

3.3 Finding the best approximation for the noisy data

From the data, we can conclude that the data can be fitted into a function of the form:

$$g(t, A, B) = AJ_2(t) + Bt \quad (12)$$

where A and B are constants that we need to find.

To find the coefficients A and B , we first try to find the mean square error between the function and the data for a range of values of A and B , which is given by:

$$\epsilon_{ij} = \frac{1}{101} \sum_{k=0}^{101} (f(t_k) - g(t_k, A_i, B_j))^2 \quad (13)$$

where ϵ_{ij} is the error for (A_i, B_j) . The contour plot of the error is shown below:

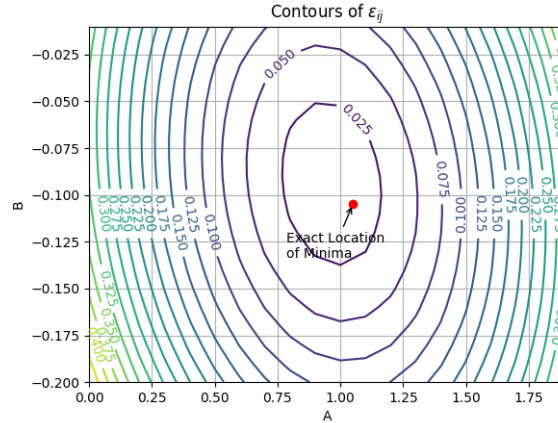


Figure 3: Contour Plot of ϵ_{ij}

We can see the location of the minima to be approximately near the original function coefficients.

Using the `lstsq` function in `scipy` package, we solve for:

$$M.p = D \quad (14)$$

where

$$M = \begin{bmatrix} J_2(t_1) & t_1 \\ \dots & \dots \\ J_2(t_m) & t_m \end{bmatrix}, p = \begin{bmatrix} A_{fit} \\ B_{fit} \end{bmatrix} \text{ and } D = \begin{bmatrix} f(t_1) \\ \dots \\ f(t_m) \end{bmatrix} \quad (15)$$

Thus, we solve for p and then find the mean square error of the values of A_{fit} and B_{fit} found using `lstsq` and the original values (1.05, -0.105).

3.4 Finding out the variation of ϵ with σ_n

We solve (14) for different values of σ_n , by changing matrix D to different columns of `fitting.dat`. We find that the variation of the mean squared error of values A_{fit} and B_{fit} is as follows:

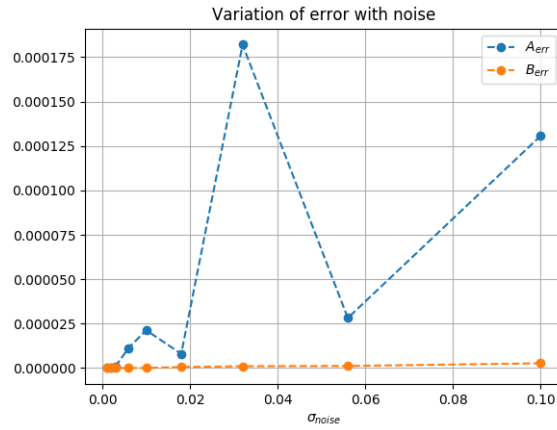


Figure 4: Mean Squared Error vs Standard Deviation

This plot does not give that much useful information between σ_n and ϵ , but when we do the `loglog` plot as below:

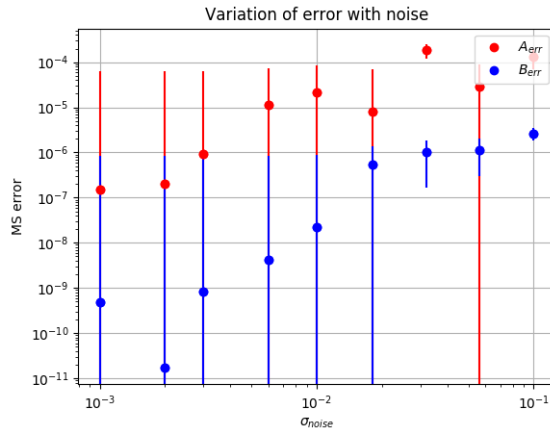


Figure 5: Error vs Standard Deviation `loglog` Plot

We can see an approximately linear relation between σ_n and ϵ . This is the required result.

4 Conclusion

From the above procedure, we were able to determine that **the logarithm of the standard deviation of the noise *linearly affects* the logarithm of the error** in the calculation of the least error fit for a given data.